Figure 21.3 The Razor Pages filter pipeline, including the five filter stages. Authorization, resource, exception, and result filters execute in exactly the same way as for the MVC pipeline. Page filters are specific to Razor Pages and execute in three places: after page hander selection, after model binding and validation, and after page handler execution.

The authorization, resource, exception, and result filters are exactly the same filters you saw for the MVC pipeline. They execute in the same way, serve the same purposes, and can be short-circuited in the same way.

NOTE These filters are literally the same classes shared between the Razor Pages and MVC frameworks.

The difference with the Razor Pages filter pipeline is that it uses page filters instead of action filters. By contrast with other filter types, page filters run three times in the filter pipeline:

- *After page handler selection*—After the resource filters have executed, a page handler is selected, based on the request's HTTP verb and the `{handler}` route value, as you learned in chapter 15. After page handler selection, a page filter method executes for the first time. You can't short-circuit the pipeline at this stage, and model binding and validation have not yet executed.
- *After model binding*—After the first page filter execution, the request is model-bound to the Razor Page's binding models and is validated. This execution is highly analogous to the action filter execution for API controllers. At this point you could manipulate the model-bound data or short-circuit the page handler execution completely by returning a different `IActionResult`.